Q3 – Resource Safety, Debugging and Robust Code

Shared: Two implementations

```
import java.io.*;
import java.util.*;
import java.util.logging.*;

public class SalesReport {
    static final Logger LOG = Logger.getLogger("report");

    // Version 1: manual close
    static double manual(List<String> files, String out) throws
IOException {
        double total = 0;
        PrintWriter w = null;
        try {
            w = new PrintWriter(new FileWriter(out));
            for (String f : files) {
                BufferedReader r = null;
                try {
                    r = new BufferedReader(new FileReader(f));
                    String line;
                    while ((line = r.readLine()) != null) total +=
Double.parseDouble(line.trim());
                } catch (IOException | NumberFormatException e) {
                    LOG.warning("Failed on file: " + f + " -> " + e);
                    throw e;
                } finally {
                    if (r != null) try { r.close(); } catch
(IOException ignore) {}
                }
            }
            w.println("TOTAL=" + total);
        } finally {
            if (w != null) w.close();
        }
        return total;
    }

    // Version 2: try-with-resources
    static double twr(List<String> files, String out) throws
IOException {
        double total = 0;
        try (PrintWriter w = new PrintWriter(new FileWriter(out))) {
            for (String f : files) {
                try (BufferedReader r = new BufferedReader(new
FileReader(f))) {
```

```
                    String line;
                    while ((line = r.readLine()) != null) total +=
Double.parseDouble(line.trim());
                } catch (IOException | NumberFormatException e) {
                    LOG.warning("Failed on file: " + f + " -> " + e);
                    throw e;
                }
            }
            w.println("TOTAL=" + total);
        }
        return total;
    }

    public static void main(String[] args) throws Exception {
        // create store1.txt, store2.txt (numbers), store3.txt
(contains "oops")
        List<String> files = List.of("store1.txt", "store2.txt",
"store3.txt");
        try { System.out.println(manual(files, "m.txt")); } catch
(Exception e) { System.out.println("manual failed: " + e); }
        try { System.out.println(twr(files, "t.txt")); }    catch
(Exception e) { System.out.println("twr failed: " + e); }
    }
}
```

Verify: with all good files, `m.txt` and `t.txt` must contain the same total (use `diff m.txt t.txt`).

Note: a version where the reader is opened but not closed on failure (e.g. calling `close()` only at the end of the try block, not in `finally`) is what leaks. Build that "leaky manual" variant to demonstrate the problem.

Student A – Debugging and experiments

- Put a breakpoint on the `parseDouble` line and in the leaky variant on the `close()` line. Step through with a corrupt line: the exception jumps over `r.close()`, so the reader stays open. (Screenshots from the IDE debugger.)
- **Failing input:** `store3.txt` containing the text `oops`, or a file that disappears midway.
- **Counting open files:** on Linux, `ls /proc/<pid>/fd | wc -l` (or `lsof -p <pid>`), or count with a wrapper class that increments on open and decrements on `close()`.
  - Leaky manual version: f–1 or more handles left open after a failure (grows with number of failures).
  - Correct `finally` or try-with-resources: 0 left open.
- **Logging:** `LOG.warning("Failed on file: " + f ...)` as shown.

Student B – Design and correctness

- **Resources:** each `FileReader/BufferedReader` per store file (closed as soon as that file is done or fails) and the `FileWriter/PrintWriter` for the summary (closed at the end, even on failure).
- **Why manual leaks:** if an exception occurs before `close()` is reached (e.g. at `parseDouble`), control leaves the block and `close()` is skipped. Also `close()` itself can throw, which can mask the original exception or skip closing the remaining resources.
- **try-with-resources guarantee:** the compiler generates an implicit `finally` that calls `close()` on every initialized resource, **in reverse order of declaration** (last opened closed first), whether the block finishes normally or by exception.
- **Suppressed exception:** if the try body throws exception A and then `close()` also throws B, A is the one propagated and B is attached to it via `addSuppressed()`. Retrieve it with `e.getSuppressed()`. (Plain try-finally would lose A and throw B.)

Joint decision

try-with-resources alone isn't enough when: (1) the failure is not about closing (bad data or corrupt records need validation, and one bad file shouldn't abort the whole report, so catch per file and continue); (2) resources are not `AutoCloseable` or are held across methods or threads; (3) partial output must be rolled back (write to a temp file and rename atomically); (4) you need retries, timeouts or limits on the number of open files; (5) the JVM dies or the disk is full (needs a flush/fsync policy).

Alternative and complexity

- **finally + close():** works but verbose and error-prone (nested try, null checks, masked exceptions).
- **No closing:** handles stay open until GC finalization/cleaner (not guaranteed), eventually causing "Too many open files".
- **Time:** reading f files with total N lines is O(N) in all three versions. Closing adds O(f), negligible.
- **Space (max handles open on failure):**
  - try-with-resources: at most 2 (current reader + writer), and 0 after unwinding.
  - correct finally: also ≤ 2 and 0 afterwards.
  - leaky or no close: up to f + 1 (grows with each file).